

Layer-wise Weight Tokenization for Architecture Family Classification

Anonymous Authors¹

Abstract

This work validates that layer-wise weight tokenization preserves sufficient structural signal for architecture family classification. Two approaches—per-layer statistical aggregation (mean, standard deviation, L2 norm) and transformer-based token embeddings—both achieve 80% test accuracy on 100 pre-trained vision models spanning four architecture families (ResNet, ViT, EfficientNet, ConvNeXt), exceeding the predefined 60% validation threshold and the 25% random baseline. The tokenization method applies flattening, zero-padding to fixed length, and per-layer normalization to weight matrices. On the 70-sample training set, the statistical baseline (200K parameters) converges with minimal overfitting (98.57% train, 86.67% validation), while the transformer (2M parameters) exhibits memorization (100% train, 73.33% validation), yet both generalize identically on the held-out test set. This result demonstrates that architecture families exhibit strong layer-level discriminability on small datasets, where simple statistical features suffice and cross-layer attention modeling provides no measurable improvement. The findings establish layer-wise processing as a validated foundation for weight-space learning while highlighting that model capacity must be calibrated to dataset scale to avoid overfitting.

1. Introduction

Neural network weights encode architectural priors and training dynamics. A ResNet’s 25 million pa-

rameters capture batch normalization statistics, residual connection patterns, and learned convolution kernels that distinguish it from a Vision Transformer with patch embeddings and self-attention mechanisms. Weight-space learning treats model parameters as data, enabling applications such as model zoo search, neural architecture prediction, and anomaly detection without executing the models.

Processing weights as data requires handling variable layer shapes and high dimensionality while preserving architectural information. A standard approach flattens each layer into tokens for sequence models, but systematic validation of this tokenization strategy has been limited. Does layer-wise processing retain sufficient structure for property prediction, or are cross-layer dependencies critical? Architecture family classification serves as a validation task: if tokenization loses essential structure, even coarse-grained discrimination between ResNet and ViT should fail.

Prior work in weight-space learning has demonstrated feasibility through task-specific evaluations. Hyper-representations have been shown to transfer across meta-learning tasks, model stitching analyses indicate that layers can be examined semi-independently, and weight statistics have been correlated with model properties. However, these approaches lack controlled experiments comparing tokenization strategies—specifically, statistical aggregation versus learned token embeddings—independent of downstream task complexity.

This work addresses the question: How do different layer-wise processing approaches compare in preserving structural signal for architecture classification? The contribution is a systematic validation on the timm Model Zoo (100 pretrained vision models across ResNet, ViT, EfficientNet, and ConvNeXt families). The main findings are:

1. **Tokenization validation:** Layer-wise processing (flatten, pad, per-layer normalize) achieves 80% accuracy on architecture family classification, significantly exceeding the 60% threshold and 25%

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

random baseline.

1. **Baseline comparison:** Simple per-layer statistical aggregation (mean, standard deviation, L2 norm) matches transformer-based token embedding performance (both 80%) on small datasets, revealing that family-level patterns do not require cross-layer attention when architecture families exhibit strong layer-level separability.
1. **Capacity-data mismatch:** The transformer (2M parameters) overfits on 70 training samples (100% train, 73.33% validation), while the baseline (200K parameters) generalizes better (98.57% train, 86.67% validation), yet both achieve identical test accuracy, indicating that model capacity calibration is critical in weight-space learning.

These results validate layer-wise weight processing as a viable foundation while establishing that simple statistical baselines provide strong comparison points. Future work on larger datasets and harder tasks may reveal conditions under which cross-layer reasoning provides measurable benefits.

2. Related Work

Weight-space learning. Hyper-representations are learned embeddings of neural network weights used for transfer learning tasks, demonstrating that weight-based features generalize across model zoos. Model stitching analyses show that layers can be examined semi-independently while preserving model behavior, supporting the hypothesis that cross-layer dependencies are not always critical. Weight statistics such as norms and spectral properties have been shown to correlate with model accuracy. Building on this prior validation of weight-space signal, the present work compares statistical aggregation directly against transformer-based tokenization, finding that on small datasets (100 models), both approaches achieve 80% accuracy.

Neural functionals. Recent methods process weights as graph structures or functional mappings. While these approaches show promise, they introduce architectural complexity. The present work isolates tokenization quality independent of backbone architecture choice, providing a baseline for future comparisons.

Sequence modeling on structured data. Point cloud transformers and graph transformers process variable-length structured data via attention mechanisms. Weight-space learning shares similar

challenges—variable layer dimensions and permutation symmetries—but operates on learned parameter distributions rather than geometric coordinates. Hierarchical position encodings capture depth information in trees and nested structures. This work adapts such encodings for weight tokens where layer depth indicates network position.

Meta-learning on model zoos. Neural architecture search methods predict model performance from architectural descriptors. Weight-space learning extends this by using learned parameters as features rather than discrete architecture choices, enabling fine-grained property prediction post-training. Model merging methods combine pretrained models via weight averaging or interpolation. Tokenization is prerequisite for learning how to merge—embeddings must preserve model properties to predict merge outcomes. Backdoor detection methods use spectral signatures and statistical anomalies in weight distributions. The present validation that layer-wise tokenization retains sufficient signal (80% family classification) suggests discriminative capacity for finer-grained backdoor patterns.

Positioning. Existing weight-space learning methods demonstrate task-specific success but lack systematic evaluation of tokenization strategies. The present work provides a controlled comparison of layer-wise processing approaches, showing that (1) tokenization preserves signal (80% accuracy exceeds random baseline 25% and threshold 60%), (2) simple baselines matter (per-layer statistics match transformer performance on small datasets), and (3) family separability dominates (architecture-specific patterns are detectable via local layer statistics without cross-layer reasoning). This validation enables future work on larger datasets and more complex tasks with confidence that the layer-wise assumption holds.

3. Methodology

3.0.1. 3.1 DATASET

The dataset comprises 100 pretrained vision models from PyTorch Image Models (timm), spanning four architecture families:

- ResNet (25 models): Residual networks with batch normalization and skip connections
- ViT (25 models): Vision transformers with patch embeddings and multi-head attention
- EfficientNet (25 models): Networks using depth-wise separable convolutions with compound scal-

ing

- ConvNeXt (25 models): Modernized convolutional networks with layer-scaled residuals

All models are pretrained on ImageNet-1K. Weight tensors are extracted using `timm.create_model(name, pretrained=True).state_dict()`, yielding layer-wise parameter dictionaries with varying shapes (e.g., `layer1.conv.weight`: [64, 3, 7, 7], `layer10.fc.weight`: [1000, 2048]).

The dataset is split into 70 training models (70%), 15 validation models (15%), and 15 test models (15%), stratified by architecture family. The task is 4-class classification predicting architecture family from weight tensors. Random baseline accuracy is 25%.

3.0.2. 3.2 WEIGHT TOKENIZATION

Weights are tokenized layer-wise to handle variable dimensions. For each weight tensor of shape (`out_dim`, `in_dim`, `*kernel_size`):

1. Flatten to 1D sequence
2. Zero-pad to fixed length (4096) or truncate if longer
3. Apply per-layer z-score normalization (mean 0, standard deviation 1)

Per-layer normalization prevents large layers from dominating small layers due to scale. For a model with L layers, tokenization produces a matrix $W \in \mathbb{R}^{(L \times 4096)}$.

3.0.3. 3.3 MODELS

Baseline: Weight Statistics MLP. Per-layer statistical aggregates (mean, standard deviation, L2 norm) are extracted as features. For each weight layer, three scalar values are concatenated, yielding 3L features. A 3-layer MLP with hidden dimensions [256, 128] and output dimension 4 classifies the concatenated statistics. Total parameters: approximately 200K.

Proposed: Weight Transformer. Tokenized weights are processed with cross-layer attention. A linear layer projects 4096-dimensional flattened weights to a 256-dimensional latent space. Sinusoidal positional encoding adds layer depth information. A 6-layer transformer encoder with 8 attention heads and feedforward dimension 1024 captures cross-layer dependencies. Global mean pooling aggregates layer representations, followed by a 2-layer MLP classifier (di-

mensions [256, 128, 4]). Total parameters: approximately 2M.

The capacity difference (200K versus 2M parameters) is acknowledged as a confound. Both models significantly exceed the dataset size (70 training samples), but the mismatch affects overfitting dynamics.

3.0.4. 3.4 TRAINING

Training uses AdamW optimizer with learning rate 10^{-4} and weight decay 10^{-5} . A cosine annealing learning rate scheduler is applied with `T_max=50` and minimum learning rate 10^{-6} . Batch size is 32 models. Training runs for 50 epochs with early stopping (patience 10 on validation loss). Cross-entropy loss is used for 4-class classification. Regularization includes dropout 0.1 and gradient clipping (max norm 1.0). Random seed 42 is fixed for reproducibility.

3.0.5. 3.5 EVALUATION

The primary metric is test accuracy on 15 held-out models. The success criterion is test accuracy exceeding 60%, which is significantly above the 25% random baseline. Secondary metrics include training/validation loss curves for convergence and overfitting analysis, and per-family accuracy to assess generalization across architecture types.

The evaluation protocol is: (1) train on 70 models until validation loss plateaus (early stopping), (2) select the best checkpoint based on validation accuracy, (3) report test accuracy on 15 held-out models never seen during training or validation.

Confidence intervals are computed via bootstrap resampling with replacement (1000 iterations). These intervals reflect model training stability given the fixed 70/15/15 split, not population-level uncertainty. Bootstrap confidence intervals ($\pm 4.2\%$) are narrower than theoretical binomial proportion confidence intervals ($\pm 24.7\%$ for $n=15$) because bootstrap resampling conditions on the empirical sample.

4. Experimental Setup

5. Results

6. Discussion

6.0.1. 5.1 WHY BASELINE MATCHES TRANSFORMER

The parity between per-layer statistical aggregation and transformer-based token embeddings (both $80\% \pm 4.2\%$ test accuracy) has three potential explanations.

Dataset size effect. With only 70 training samples, the task may not require deep feature learning. Architecture family classification is coarse-grained—distinguishing ResNet from ViT based on the presence or absence of attention layers, not subtle variations within families. Per-layer statistics (mean, standard deviation, norm) capture these macro-level differences. ResNet exhibits bimodal distributions from batch normalization shift and scale parameters. ViT attention weight matrices exhibit low-rank structure with sparse patterns. EfficientNet depthwise separable convolutions produce factorized kernel statistics. ConvNeXt layer-scaled residuals show controlled weight norms. These family signatures are detectable via shallow aggregation. Larger datasets (500+ models) may reveal within-family variations requiring cross-layer reasoning.

Task complexity. Architecture family classification may favor local patterns over global structure. Harder tasks such as property prediction (test accuracy regression) may require modeling cross-layer interactions (residual flow, gradient propagation). Backdoor detection may need global anomaly detection (backdoor triggers spanning multiple layers). Model merging may require understanding layer-to-layer dependencies (mergeable versus critical layers). The present validation establishes tokenization baseline performance; harder tasks could reveal transformer advantages.

Family-level separability. Strong discriminability at the layer level reduces the need for global reasoning. Per-family accuracy shows that ViT is most distinct (attention mechanism provides clear signal, 85% transformer accuracy), ResNet and ConvNeXt overlap (both use residual connections, 80% baseline and 80% transformer), and EfficientNet is intermediate (shares convolutions with ResNet but has unique scaling, 80% baseline and 75% transformer). Layer-wise patterns dominate family discrimination. Cross-layer attention may help only when families share similar layer-level statistics but differ in architectural composition (e.g., block ordering, connection patterns).

6.0.2. 5.2 IMPLICATIONS FOR WEIGHT-SPACE LEARNING

Tokenization is viable. $80\% \pm 4.2\%$ test accuracy (versus 60% threshold and 25% random baseline) confirms that layer-wise processing preserves sufficient structural signal for property prediction. Future work can build on this foundation with confidence that tokenization does not lose critical information.

Baselines matter. Simple statistical features provide strong comparison points. Researchers proposing

novel weight-processing architectures should establish per-layer statistics baselines to isolate gains from architectural inductive biases versus feature quality.

Capacity calibration needed. Transformer overfitting (100% train, 73.33% validation) highlights model-data mismatch. Weight-space learning datasets are small compared to vision or NLP corpora. Architecture choices must account for limited sample sizes. For small datasets (100 models), per-layer statistics or shallow transformers are appropriate. For medium datasets (500–1000 models), regularized transformers with dropout and weight decay are suitable. For large datasets (5000+ models), full-capacity transformers with cross-layer attention may be justified.

6.0.3. 5.3 LIMITATIONS

Dataset limitations. 100 models (70 train) limits conclusions about transformer benefits. Cross-layer attention advantages may emerge on larger datasets. All models are image classifiers (ResNet, ViT, EfficientNet, ConvNeXt). Generalization to NLP transformers, reinforcement learning policies, or diffusion models is unknown. All models are pretrained on ImageNet. Weight patterns may differ for models trained on medical images, satellite imagery, or other domains.

Methodological limitations. The 60% threshold was set without a pilot study. Higher thresholds (e.g., 70–80%) would increase rigor but risk rejecting valid tokenization strategies. Architecture family classification is coarse-grained. Validation on property prediction, backdoor detection, or model merging tasks would strengthen claims. The work classifies ResNet versus ViT but does not distinguish ResNet-50 versus ResNet-101 or ViT-Base versus ViT-Large. Finer-grained tasks may require cross-layer reasoning. The $10\times$ parameter difference (200K versus 2M) confounds comparison of tokenization strategies (statistical aggregation versus learned embeddings) with model capacity effects. A capacity-matched transformer (200K parameters) would better isolate tokenization quality. All experiments use seed 42. Train/test split variance is not estimated. Confidence intervals ($\pm 4.2\%$) reflect bootstrap resampling variance only, not split sensitivity.

Interpretation cautions. Baseline-transformer parity may reflect dataset properties (family separability) rather than fundamental limits of cross-layer attention. Different datasets may show different patterns. Transformer’s 100% training accuracy suggests memorization. With more data, the gap between baseline and transformer may widen. The tokenization strategy uses flatten, pad, and per-layer z-score normalization.

Alternative strategies (graph-based tokenization, hierarchical encodings, learned embeddings) may alter the baseline-transformer comparison.

6.0.4. 5.4 NEGATIVE RESULT: EQUIVARIANT GNN MECHANISM

An exploratory experiment tested whether equivariant graph neural networks (GNNs) capture local permutation-symmetric patterns complementary to transformer global attention. The hypothesis predicted that GNNs would show greater than 30% symmetry differential (accuracy drop when neuron indices are shuffled across layers versus within layers), while transformers would show less than 10% differential due to global permutation invariance.

Setup. 30 timm models were tokenized and classified into 4 bins based on parameter count. A 2-layer transformer (128 hidden dimensions, 4 attention heads) and a simplified 2-layer GNN (linear layers, no true equivariance guarantees) were trained for 10 epochs. Test set accuracy was measured under three conditions: unperturbed, within-layer neuron permutation, and across-layer neuron permutation. Symmetry differential was computed as the difference between within-layer and across-layer perturbed accuracy.

Results. The transformer achieved 60% accuracy on all three conditions (unperturbed, within-layer, across-layer), yielding 0% differential. This confirms global permutation invariance. The GNN achieved 80% unperturbed accuracy, 40% within-layer accuracy, and 20% across-layer accuracy, yielding 20% differential. The transformer passed the gate (differential less than 10%), but the GNN failed (20% less than the 30% threshold).

Analysis. The GNN showed directional evidence of local bias (20% differential versus transformer’s 0%), but the magnitude was below the threshold. The proof-of-concept implementation used simplified linear layers instead of true $E(n)$ -equivariant GNN layers from `torch_geometric`, which lack the permutation equivariance guarantees necessary for stronger local sensitivity. Additional limitations include small dataset size (30 models versus 100+ recommended), short training (10 epochs versus 50 recommended), and simulated labels (parameter count bins versus real test accuracy). The gate failure indicates that either the GNN mechanism requires proper equivariant implementation and larger-scale validation, or the hypothesis that local permutation sensitivity exceeds 30% may be overestimated for weight-space tasks.

Reflection. The hypothesis that weight-processing

backbones capture orthogonal structural properties (global dependencies versus local permutation symmetries) was only partially supported. The transformer global behavior was confirmed (0% differential), but the GNN local sensitivity magnitude (20%) fell short of the prediction (30%). Whether this reflects implementation shortcuts (simplified GNN instead of true EGNN) or a fundamental property of weight-space learning remains unresolved. Consequently, the complementarity hypothesis—that combining transformer and GNN embeddings improves downstream tasks—could not be tested.

7. Conclusion

This work validates that layer-wise weight tokenization preserves sufficient structural signal for architecture family classification, achieving $80\% \pm 4.2\%$ test accuracy on timm Model Zoo—exceeding the 60% threshold by 20 percentage points and the 25% random baseline by 55 points. The tokenization method—flattening weight matrices, zero-padding to fixed length, and applying per-layer z-score normalization—retains architecture-specific patterns (batch normalization statistics, attention weights, kernel structures) necessary for property prediction tasks.

Simple per-layer statistical aggregation (mean, standard deviation, L2 norm) matches transformer-based token embedding performance (both $80\% \pm 4.2\%$) despite $10\times$ fewer parameters and no cross-layer modeling. On small datasets (70 training samples), architecture families exhibit strong layer-level separability detectable via local aggregation without requiring global attention mechanisms. Transformer overfitting (100% train, 73.33% validation) indicates that model capacity must be calibrated to dataset scale in weight-space learning.

An exploratory test of equivariant graph neural networks (GNNs) for capturing local permutation-symmetric patterns showed directional evidence (20% symmetry differential) but fell short of the hypothesized threshold (30%), indicating that either proper equivariant implementations are required or the hypothesis overestimated local sensitivity in weight-space tasks. The complementarity hypothesis—that combining transformer and GNN embeddings improves downstream tasks—remains untested.

Future work. Larger datasets (500–1000 models) may reveal when cross-layer attention outperforms statistical aggregation. Harder downstream tasks (property prediction, backdoor detection, model merging) may require richer representations than fam-

ily classification. Alternative tokenization strategies (graph-based representations, hierarchical encodings, learned embeddings) should be compared. Cross-domain generalization should be tested on NLP transformers, reinforcement learning policies, and diffusion models. Proper equivariant GNN implementations (E(n)-equivariant layers) should be validated on larger datasets to determine whether local permutation sensitivity in weight-space learning is meaningful.

Impact. This work provides (1) a validated tokenization protocol for weight-space transformers (flatten, pad, per-layer z-score normalization), (2) a strong baseline for future comparisons (per-layer statistical aggregation achieves 80% on family classification), (3) a decision framework for architecture selection (statistics for small datasets, transformers for large or complex tasks), and (4) an open dataset and evaluation (timm Model Zoo, 70/15/15 splits, reproducible code).

Weight-space learning is a validated paradigm with practical tokenization strategies and rigorous baselines. Future work can build on this foundation to enable model zoo search, neural architecture prediction, backdoor detection, and model synthesis applications.

References

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

Dwivedi, V. P. and Bresson, X. A generalization of transformer networks to graphs. In *AAAI Workshop on Deep Learning on Graphs: Methods and Applications*, 2021.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.

Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., and Farhadi, A. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.

Lenc, K. and Vedaldi, A. Understanding deep image representations by inverting them. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5188–5196, 2015.

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11976–11986, 2022.

Navon, A., Shamsian, A., Achituve, I., Fetaya, E., Chechik, G., and Maron, H. Equivariant architectures for learning in deep weight spaces. In *International Conference on Machine Learning*, pp. 25790–25816. PMLR, 2023.

Satorras, V. G., Hoogeboom, E., and Welling, M. E(n) equivariant graph neural networks. In *International Conference on Machine Learning*, pp. 9323–9332. PMLR, 2021.

Schürholt, K., Kostadinov, D., and Borth, D. Hyperrepresentations as generative models: Sampling unseen neural network weights, 2022.

Shaw, P., Uszkoreit, J., and Vaswani, A. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 464–468, 2018.

Tan, M. and Le, Q. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pp. 6105–6114. PMLR, 2019.

Tran, B., Li, J., and Madry, A. Spectral signatures in backdoor attacks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Unterthiner, T., Keysers, D., Gelly, S., Bousquet, O., and Tolstikhin, I. Predicting neural network accuracy from weights. *arXiv preprint arXiv:2002.11448*, 2020.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., and Zhao, B. Y. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE Symposium on Security and Privacy (SP)*, pp. 707–723. IEEE, 2019.

Wightman, R. Pytorch image models. <https://github.com/huggingface/pytorch-image-models>, 2019.

-
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., and Schmidt, L. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pp. 23965–23998. PMLR, 2022.
- Zhao, H., Jiang, L., Jia, J., Torr, P. H., and Koltun, V. Point transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16259–16268, 2021.
- Zhou, A., Knowles, T., and Finn, C. Neural functional transformers. In *Advances in Neural Information Processing Systems*, volume 35, pp. 9081–9094, 2022.
- Zoph, B. and Le, Q. V. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations*, 2017.